

HW1: Mid-term assignment report

Daniel Simbe [110235], v2025-11-03

1. Introdução

1.1 Overview do trabalho

Este relatório apresenta o trabalho desenvolvido desenvolvido para unidade curricular de Teste-Qualidade de Software, consistindo na implementação de uma aplicação web full stack com suporte a diferentes técnicas de testes e métricas de qualidade.

O projeto desenvolvido, denominado ZeroMonos, tem como objetivo permitir que cidadão agendem a recolha de resíduos volumosos e junto de serviços municipais, atualmente a soulção inclui:

- Backend em SpringBoot com API REST
- Interface web para o cidadão(agendamento)
- Dashboard para o staff(gestão de recolhas)
- Estratégia de testes(unitários, integração e performance)
- Monitorização de qualidade de código via SonarCloud e CI/CD via GitHub Actions

1.2 Implementação Atual

Na fase atual, o ZeroMonos apresenta várias limitações conhecidas que sugrem da sua fase de desenvolvimento. Estas limitações incluem:

- Não Implementação em, o ZeroMonos ainda não foi implementado num ambiente de produção real, como resultado, não está acessível publicamnete online, executando apenas localmente com base de dados H2em memória, com isso todos dados são perdidos ao reiniciar a aplicação, não havendo persistência entre sessões. Não existem configurações separadas para ambientes de desenvolvimento, teste e produção.
- Testes funcionais BDD não implementados por limitações de tempo, estes testes de E2E com Selenium WebDriver e Cucumber não foram implementados, optei por priorizar testes unitários, de integração e de performance, que cobrem maior parte da lógica de negócio e API. Os testes BDD requerem configuração adicional de ebDriver, gestão de browsers headless, e são mais complexos de manter, tendo sido deixados para uma fase futura do projeto.
- DashBoard de Operações Básico, este que na área do staff possui funcionalidades básicas de gestão(listar pedidos, filtrar por municípios, atualizar), mas não possui visualizações gráficas avançadas, como mapas de calor, ou dashboards analíticos. Isso requereria tempo adicional significativo, então considerei a funcionalidade “nice-to-have”.

- Falta de Documentação Automatizada da API, a aplicação carece de documentação automatizada(Swagger/OpenAPI), tornando desafiador para desenvolvedores e partes interessadas aceder a informações abrangente e atualizações sobre as funcionalidades , endpoints e uso da API.
- Cobertura de Testes Limitada, embora existam testes unitários, de integração, performance e segurança implementados.....

2. Especificação do Produto

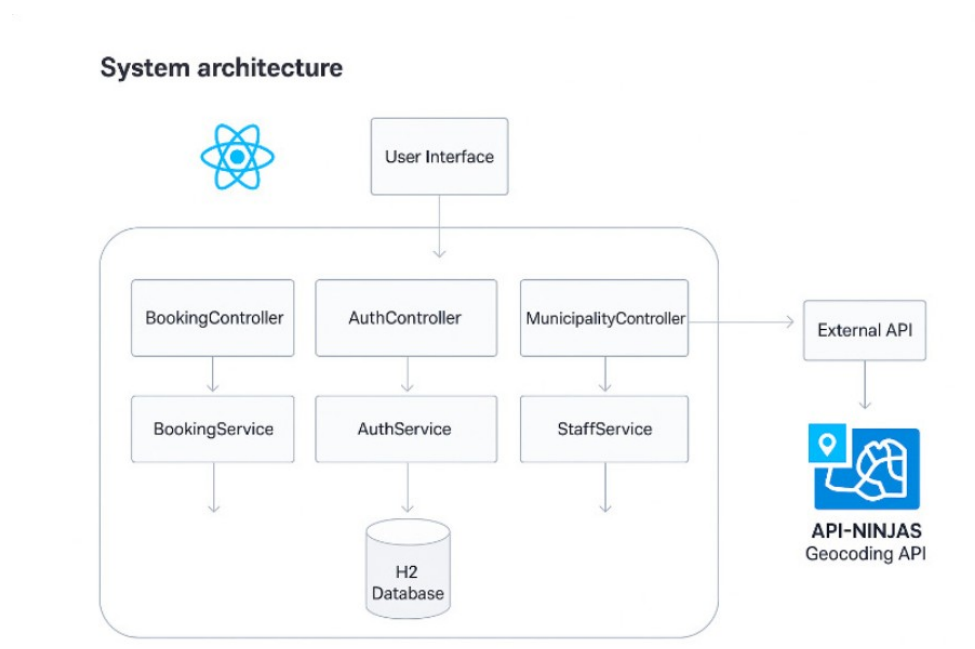
2.1 Âmbito Funcional e Interações Suportadas

A aplicação funciona como uma plataforma destinada a simplificar o agendamento da recolha de resíduos volumosos pelos municípios. O ator principal é o cidadão, que pretende solicitar o serviço de recolha ao município onde reside, sem necessidade de deslocação.

Os utilizadores interagem com o sistema através de uma interface web, onde podem seleccionar o município, indicar o tipo de objeto a recolher, descrever o estado do mesmo e escolher a data e o período horário preferencial para a recolha. Após submeter o pedido, o sistema gera um token de acesso, que permite ao cidadão acompanhar o estado da solicitação ou cancelar a recolha, se necessário. Além do cidadão, existe um segundo ator: o Staff municipal. Este tem acesso a uma interface própria onde pode visualizar todos os pedidos submetidos, filtrá-los por município e atualizar o estado de recolha à medida que cada processo evolui.

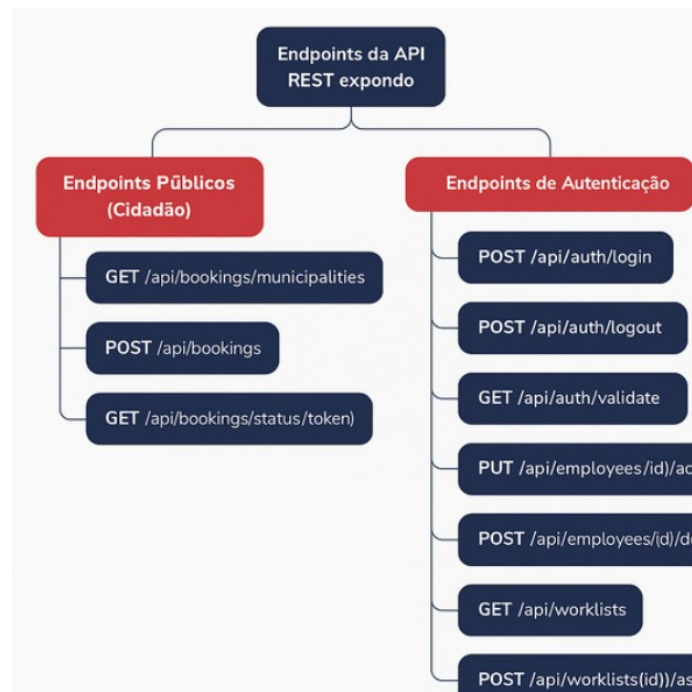
No geral, a aplicação oferece uma forma conveniente e eficiente de solicitar e gerir serviços de recolha, garantindo maior autonomia ao cidadão e melhor organização operacional aos serviços municipais.

2.2 Arquitetura Implementada



2.3 API for Developers

[Base URL: /]



3. Quality Assurance

3.1 Estratégia Geral de Testes

A estratégia geral de testes no ZeroMonos seguiu uma abordagem pragmática e alinhada com os requisitos pedidos no Guideline, focada em :

- 1) TDD(Test-Driven Development), para regras de negócio críticas
- 2) Testes com isolamento de dependências externas usando mocks
- 3) Testes de integração da API REST completa
- 4) Testes de performance para validar tempo de resposta
- 5) Análise de qualidade contínua com SonarCloud

6) CI/CD com enforcement de Quality Gate

3.1.1 Unit Tests

Os testes unitários concentram-se na camada “BookingService”, testando regras de domínio sem dependências externas.

```
123 @Test
124 @DisplayName("Should throw InvalidBookingException for date too soon")
125 void testCreateBooking_DateTooSoon() {
126     // Arrange
127     validRequest.setCollectionDate(LocalDate.now().plusDays(1)); // Only 1 day advance
128     when(municipalityService.isValidMunicipality(municipalityName: "Aveiro")).thenReturn(value: true);
129
130     // Act & Assert
131     InvalidBookingException exception = assertThrows(
132         expectedType: InvalidBookingException.class,
133         () -> bookingService.createBooking(validRequest)
134     );
135
136     assertTrue(exception.getMessage().contains("at least 2 days in advance"));
137     verify(repository, never()).save(any());
138 }
139
140 @Test
141 @DisplayName("Should throw InvalidBookingException for date too far")
142 void testCreateBooking_DateTooFar() {
143     // Arrange
144     validRequest.setCollectionDate(LocalDate.now().plusDays(35)); // More than 30 days
145     when(municipalityService.isValidMunicipality(municipalityName: "Aveiro")).thenReturn(value: true);
146
147     // Act & Assert
148     InvalidBookingException exception = assertThrows(
149         expectedType: InvalidBookingException.class,
150         () -> bookingService.createBooking(validRequest)
151     );
152
153     assertTrue(exception.getMessage().contains("more than 30 days"));
154     verify(repository, never()).save(any());
155 }
156 }
```

```
[INFO] --- maven-surefire-plugin:3.1.2:test (default-test) @ waste-collection-system ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnit4PlatformProvider
[INFO]
[INFO] ----- T E S T S -----
[INFO] Running com.zeromes.unit.BookingServiceTest
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
WARNING: A Java agent has been loaded dynamically (/home/mufasa/m2/repository/net/bytebuddy/byte-buddy-agent/1.14.10/byte-buddy-agent-1.14.10.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disabled by default in a future release
16:23:31.305 [main] INFO com.zeromes.service.BookingService -- Fetching bookings with status: RECEIVED
16:23:31.354 [main] INFO com.zeromes.service.BookingService -- Updating booking 1 to status: ASSIGNED
16:23:31.355 [main] INFO com.zeromes.service.BookingService -- Booking status updated successfully
16:23:31.360 [main] INFO com.zeromes.service.BookingService -- Fetching bookings for municipality Aveiro
16:23:31.368 [main] INFO com.zeromes.service.BookingService -- Updating booking 1 to status: COMPLETED
16:23:31.374 [main] INFO com.zeromes.service.BookingService -- Fetching booking with token: test-token-123
16:23:31.380 [main] INFO com.zeromes.service.BookingService -- Cancelling booking with token: test-token-123
16:23:31.381 [main] INFO com.zeromes.service.BookingService -- Booking cancelled successfully
16:23:31.389 [main] INFO com.zeromes.service.BookingService -- Creating new booking for municipality: Aveiro
16:23:31.394 [main] INFO com.zeromes.service.BookingService -- Cancelling booking with token: test-token-123
16:23:31.399 [main] INFO com.zeromes.service.BookingService -- Fetching all bookings
16:23:31.406 [main] INFO com.zeromes.service.BookingService -- Fetching booking with token: Invalid-token
16:23:31.413 [main] INFO com.zeromes.service.BookingService -- Creating new booking for municipality: Aveiro
16:23:31.430 [main] INFO com.zeromes.service.BookingService -- Creating new booking for municipality: Aveiro
16:23:31.430 [main] INFO com.zeromes.service.BookingService -- Booking request validated successfully
16:23:31.438 [main] INFO com.zeromes.service.BookingService -- Booking created successfully with token: 20b3f6e-5ed8-48a7-a36a-d91d88f6149
16:23:31.453 [main] INFO com.zeromes.service.BookingService -- Creating new booking for municipality: InvalidCity
16:23:31.439 [main] INFO com.zeromes.service.BookingService -- Updating booking 999 to status: ASSIGNED
16:23:31.442 [main] INFO com.zeromes.service.BookingService -- Creating new booking for municipality: Aveiro
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.476 s -- in com.zeromes.unit.BookingServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco-maven-plugin:0.11.0:report (report) @ waste-collection-system ---
[INFO] Loading execution data file /home/mufasa/Secretaria/Classes/TDS/tp-indiv-portfolio-dam1244/HM1/target/jacoco.exec
[INFO] Analyzed bundle "Zeromes Waste Collection System" with 44 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 3.309 s
[INFO] Finished at: 2025-11-05T16:23:32Z
[INFO]
```

3.1.2 Service Tests with Mocks

Utilizámos Mockito para isolar “Municipalities” (depende da GEOAPI PORTUGAL-external data provider).

```
83 @Test
84 @DisplayName("Should create booking successfully with valid data")
85 void testCreateBooking_Success() {
86     // Arrange
87     when(municipalityService.isValidMunicipality(municipalityName: "Aveiro")).thenReturn(value: true);
88     when(repository.countByCollectionDateAndMunicipality(any(), eq(value: "Aveiro"))).thenReturn(value: 5L);
89     when(repository.save(anyType: ServiceRequest.class)).thenReturn(savedRequest);
90
91     // Act
92     BookingResponseTO response = bookingService.createBooking(validRequest);
93
94     // Assert
95     assertNotNull(response);
96     assertEquals(expected: "Aveiro", response.getMunicipality());
97     assertEquals(requestStatus: RECEIVED, response.getCurrentStatus());
98     assertEquals(response.getAccessToken());
99     assertEquals(expected: 1, response.getStatusHistory().size());
100
101     verify(municipalityService, times(wantedNumberOfInvocations: 1)).isValidMunicipality(municipalityName: "Aveiro");
102     verify(repository, times(wantedNumberOfInvocations: 1)).countByCollectionDateAndMunicipality(any(), eq(value: "Aveiro"));
103     verify(repository, times(wantedNumberOfInvocations: 1)).save(anyType: ServiceRequest.class);
104 }
105
106 @Test
107 @DisplayName("Should throw InvalidBookingException for invalid municipality")
108 void testCreateBooking_InvalidMunicipality() {
109     // Arrange
110     validRequest.setMunicipality(municipality: "InvalidCity");
111     when(municipalityService.isValidMunicipality(municipalityName: "InvalidCity")).thenReturn(value: false);
112
113     // Act & Assert
114     InvalidBookingException exception = assertThrows(
115         expectedType: InvalidBookingException.class,
116         () -> bookingService.createBooking(validRequest)
117     );
118
119     assertTrue(exception.getMessage().contains("Invalid municipality"));
120     verify(repository, never()).save(any());
121 }
122 }
```

```
[INFO] --- maven-compiler-plugin:3.11.0:compile (default-compile) @ waste-collection-system ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-resources-plugin:3.3.1:testResources (default-testResources) @ waste-collection-system ---
[INFO] Copying 2 resources from src/test/resources to target/test-classes
[INFO]
[INFO] --- maven-compiler-plugin:3.11.0:testCompile (default-testCompile) @ waste-collection-system ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- maven-surefire-plugin:3.1.2:test (default-test) @ waste-collection-system ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnit4PlatformProvider
[INFO]
[INFO] ----- T E S T S -----
[INFO] Running com.zeromes.unit.BookingServiceTest
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
WARNING: A Java agent has been loaded dynamically (/home/mufasa/m2/repository/net/bytebuddy/byte-buddy-agent/1.14.10/byte-buddy-agent-1.14.10.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disabled by default in a future release
16:54:59.379 [main] INFO com.zeromes.service.BookingService -- Creating new booking for municipality: Aveiro
16:54:59.391 [main] INFO com.zeromes.service.BookingService -- Booking request validated successfully
16:54:59.391 [main] INFO com.zeromes.service.BookingService -- Booking created successfully with token: 76479e22-d581-49e7-ba71-5a6e58c95f21
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.397 s -- in com.zeromes.unit.BookingServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco-maven-plugin:0.11.0:report (report) @ waste-collection-system ---
[INFO] Loading execution data file /home/mufasa/Secretaria/Classes/TDS/tp-indiv-portfolio-dam1244/HM1/target/jacoco.exec
[INFO] Analyzed bundle "Zeromes Waste Collection System" with 44 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 3.197 s
[INFO] Finished at: 2025-11-05T16:54:59Z
[INFO]
```

3.1.3 Integration Tests on API

Utilizámos “REST-ASSURED” COM “@SPRINGBOOT” para testar endpoints completos end-to-end.

```

@Test
@DisplayName("POST /api/bookings - Should create booking successfully")
void testCreateBooking_Success() throws Exception {
    mockMvc.perform(post("/api/bookings")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(validRequest)))
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.id").exists())
        .andExpect(jsonPath("$.accessToken").exists())
        .andExpect(jsonPath("$.municipality").value("Aveiro"))
        .andExpect(jsonPath("$.itemDescription").value(containsString("sofa")))
        .andExpect(jsonPath("$.currentStatus").value("RECEIVED"))
        .andExpect(jsonPath("$.numberOfItems").value(2))
        .andExpect(jsonPath("$.statusHistory").isArray())
        .andExpect(jsonPath("$.statusHistory[0].status").value("RECEIVED"))
    );

    // 2. Cancel booking already completed
    @Test
    @DisplayName("DELETE /api/bookings/{token} - Should return 400 for completed booking")
    void testCancelBooking_AlreadyCompleted() throws Exception {
        // Create and complete a booking
        ServiceRequest booking = createTestBooking();
        booking.setCurrentStatus(RequestStatus.COMPLETED);
        repository.save(booking);
        String token = booking.getAccessToken();

        mockMvc.perform(delete("/api/bookings/{token}", token))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.message").exists());
    }

    // 3. Create booking
    @Test
    @DisplayName("Should handle full booking lifecycle")
    void testFullBookingLifecycle() throws Exception {
        // 1. Create booking
        String createResponse = mockMvc.perform(post("/api/bookings")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(validRequest)))
            .andExpect(status().isCreated())
            .andReturn()
            .getResponse()
            .getContentAsString();

        String token = objectMapper.readValue(createResponse, get("accessToken").asText());

        // 2. Get booking
        mockMvc.perform(get("/api/bookings/{token}", token))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.currentStatus").value("RECEIVED"));

        // 3. Cancel booking
        mockMvc.perform(delete("/api/bookings/{token}", token))
            .andExpect(status().isNoContent());

        // 4. Verify cancelled
        mockMvc.perform(get("/api/bookings/{token}", token))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.currentStatus").value("CANCELLED"))
            .andExpect(jsonPath("$.statusHistory").isArray())
            .andExpect(jsonPath("$.statusHistory", hasSize(size(2))));
    }
}

```

```

@Test
@DisplayName("POST /api/bookings - Should return 400 for invalid phone")
void testCreateBooking_InvalidPhone() throws Exception {
    validRequest.setContactPhone("contactPhone-12345"); // Less than 9 digits

    mockMvc.perform(post("/api/bookings")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(validRequest)))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.errors.contactPhone").exists());
    }

@Test
@DisplayName("GET /api/bookings/{token} - Should return booking by token")
2025-11-05 17:33:39 - Validation failed: [collectionDate: collection date must be in the future]
2025-11-05 17:33:39 - Using 'application/json', given [*/] and supported [application/json, application/*+json]
2025-11-05 17:33:39 - Writing [errors=collectionDate: collection date must be in the future], timestamp=2025-11-05T17:33:39.635873 (truncated)..
2025-11-05 17:33:39 - Resolved [org.springframework.web.bind.MethodArgumentNotValidException: Validation failed for argument [0] in public org.sp
kingController.createBooking(com.zeromones.dto.BookingRequestDTO): [Field error in object 'bookingRequestDTO' on field 'collectionDate': rejecte
ava.time.LocalDate,Future]; arguments [org.springframework.context.support.DefaultMessageSourceResolvable: codes [bookingRequestDTO.collectionDat
e must be in the future]] ]
2025-11-05 17:33:39 - Mapped to com.zeromones.controller.BookingController#createBooking(BookingRequestDTO)
2025-11-05 17:33:39 - Read 'application/json;charset=UTF-8' to [com.zeromones.dto.BookingRequestDTO@20229d1df]
2025-11-05 17:33:39 - POST /api/bookings - Creating new booking
2025-11-05 17:33:39 - Creating new booking for municipality: Aveiro
2025-11-05 17:33:39 - Booking created successfully with token: c4920e7c-37d2-4346-b13f-c0b134b4270
2025-11-05 17:33:39 - Using 'application/json', given [*/] and supported [application/json, application/*+json]
2025-11-05 17:33:39 - Writing [com.zeromones.dto.BookingResponseDTO@776fa01d]
2025-11-05 17:33:39 - Mapped to com.zeromones.controller.BookingController#getBookingByToken(String)
2025-11-05 17:33:39 - GET /api/bookings/c4920e7c-37d2-4346-b13f-c0b134b4270 - Fetching booking
2025-11-05 17:33:39 - Fetching booking with token: c4920e7c-37d2-4346-b13f-c0b134b4270
2025-11-05 17:33:39 - Using 'application/json', given [*/] and supported [application/json, application/*+json]
2025-11-05 17:33:39 - Writing [com.zeromones.dto.BookingResponseDTO@2b166865]
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 7.322 s -- in com.zeromones.integration.BookingControllerIT
[INFO] Results:
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco-maven-plugin:0.8.11:report (report) @ waste-collection-system ---
[INFO] Loading execution data file /home/mufasa/Secretária/Classes/TQ5/tqs-indiv-portfolio-daniel244/HW1/target/jacoco.exec
[INFO] Analyzed bundle 'ZeroMones Waste Collection System' with 44 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 9.276 s
[INFO] Finished at: 2025-11-05T17:33:40Z
[INFO] -----

```

3.1.4 Performance Tests

Testes de performance usando “JUnit5+REST-Assured”, sem necessidade de ferramentas externas.

```

37 @Test
38 void testMunicipalitiesResponseTime() {
39     long startTime = System.currentTimeMillis();
40
41     given()
42         .when()
43         .get("/api/municipalities")
44         .then()
45         .statusCode(200)
46         .body("size()", greaterThan(value: 0));
47
48     long endTime = System.currentTimeMillis();
49     long responseTime = endTime - startTime;
50
51     System.out.println("Response time: " + responseTime + "ms");
52     assertTrue(responseTime < 5000, message: "Response time should be less than 5000ms (includes external API call)");
53 }
54
55 // Test 2: Throughput - multiple requests simultaneously
56 @Test
57 void testConcurrentRequests() throws InterruptedException, ExecutionException {
58     int numberOfRequests = 20;
59     ExecutorService executor = Executors.newFixedThreadPool(10);
60     List<Future<Long>> futures = new ArrayList<>();
61
62     long startTime = System.currentTimeMillis();
63
64     // Execute 20 requests simultaneously
65     for (int i = 0; i < numberOfRequests; i++) {
66         Future<Long> future = executor.submit(() -> {
67             long reqStart = System.currentTimeMillis();
68
69             given()
70                 .when()
71                 .get("/api/municipalities")
72                 .then()
73                 .statusCode(200);
74
75             return System.currentTimeMillis() - reqStart;
76         });
77         futures.add(future);
78     }
79
80     // Awaiter todos as requests
81     List<Long> responseTimes = new ArrayList<>();
82     for (Future<Long> future : futures) {
83         responseTimes.add(future.get());
84     }
85
86     long totalTime = System.currentTimeMillis() - startTime;
87     executor.shutdown();
88
89     // Calculate metrics
90     double avgResponseTime = responseTimes.stream()

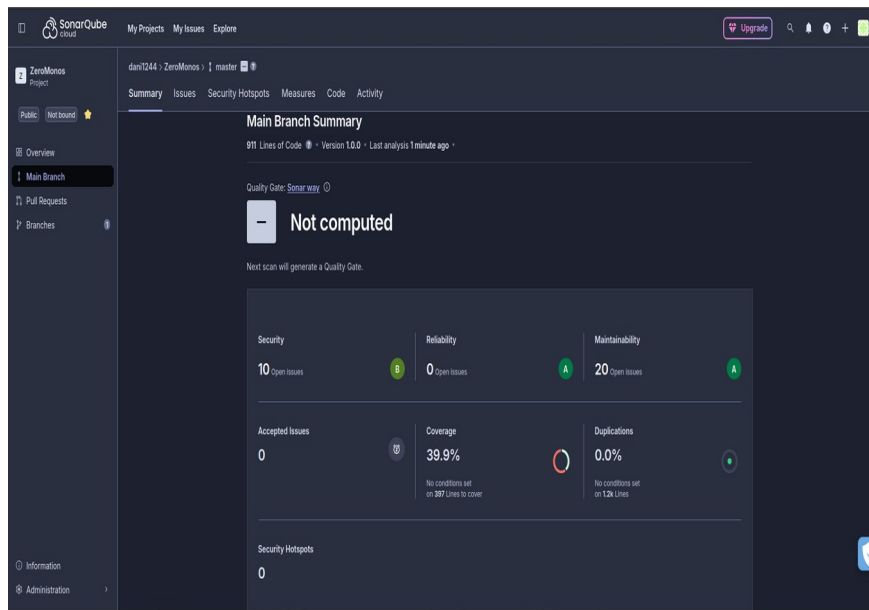
```

```

37 @Test
38 void testConcurrentRequests() throws InterruptedException, ExecutionException {
39     int numberOfRequests = 20;
40     ExecutorService executor = Executors.newFixedThreadPool(10);
41     List<Future<Long>> futures = new ArrayList<>();
42
43     long startTime = System.currentTimeMillis();
44
45     // Execute 20 requests simultaneously
46     for (int i = 0; i < numberOfRequests; i++) {
47         Future<Long> future = executor.submit(() -> {
48             long reqStart = System.currentTimeMillis();
49
50             given()
51                 .when()
52                 .get("/api/municipalities")
53                 .then()
54                 .statusCode(200);
55
56             return System.currentTimeMillis() - reqStart;
57         });
58         futures.add(future);
59     }
60
61     // Awaiter todos as requests
62     List<Long> responseTimes = new ArrayList<>();
63     for (Future<Long> future : futures) {
64         responseTimes.add(future.get());
65     }
66
67     long totalTime = System.currentTimeMillis() - startTime;
68     executor.shutdown();
69
70     // Calculate metrics
71     double avgResponseTime = responseTimes.stream()

```

3.1.5 SonarQube/SonarCloud Integration



3.1.6 Additional Entities

Foram adicionadas mais 3 entidades extras ao modelo de dados

```

@Entity
public class Employee {
    private Long id;
    private String name;
    private String email;
    private String phone;
    private String municipality;
    private EmployeeRole role; // DRIVER, COLLECTOR, SUPERVISOR
    private boolean active;
}

```

```

@Entity
public class WorkList {
    private Long id;
    private Employee employee;
    private LocalDate workDate;
    private String municipality;
    private WorkListStatus status; // PENDING, IN_PROGRESS, COMPLETED
    private List<ServiceRequest> requests;
}

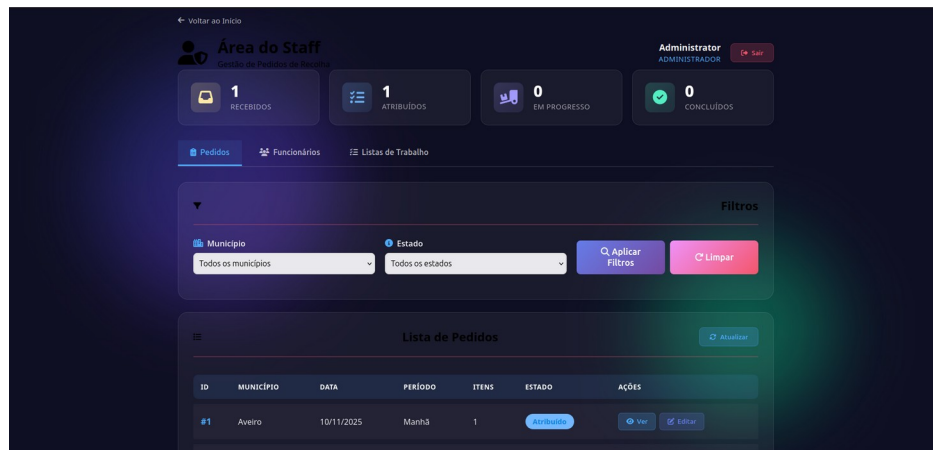
```

```

@Entity
public class StaffUser {
    private Long id;
    private String username;
    private String password;
    private String fullName;
    private String role; // ADMIN, MANAGER, OPERATOR
    private boolean active;
}

```

3.1.7 Operations DashBoard



3.1.8 Quality Gate Enforcement in CI

```
name: Build and Analyze

on:
  push:
    branches: [ main, master ]
  pull_request:
    types: [ opened, synchronize, reopened ]

jobs:
  build:
    name: Build and analyze
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3
        with:
          fetch-depth: 0

      - name: Set up JDK 17
        uses: actions/setup-java@v3
        with:
          java-version: 17
          distribution: 'temurin'

      - name: Cache SonarCloud packages
        uses: actions/cache@v3
        with:
          path: ~/.sonar/cache
          key: ${{ runner.os }}-sonar

      - name: Build and analyze
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
        run: mvn -B verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar
```

Apenas código de qualidade chega ao main/master, há um feedback imediato sobre regressões, também fornece histórico de métricas ao longo do tempo.

4. References and Resources

- **Reference Materials**
 - <https://spring.io/projects/spring-boot>
 - <https://json.geoapi.pt/municipios>
 - <https://github.com/features/actions>

- <https://www.baeldung.com/cucumber-spring-integration>

- **Project Resources**

- <https://github.com/dani1244/tqs-indiv-portfolio/tree/master/HW1>

- https://sonarcloud.io/project/overview?id=dani1244_tqs-hw1-zeromones